

3. C++ Vectors and Writing Output to File

3.1	Motivating application: Heat fin with inconsistent boundary conditions	26
3.2	Code overview	27
3.3	C++ vectors	28
3.4	Writing data to a file	30
3.5	Homework	31

Introduction

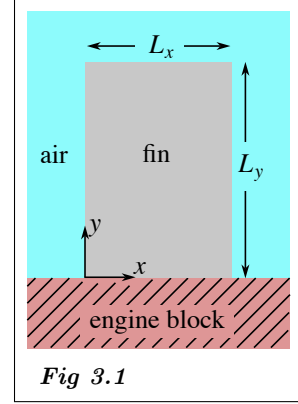
This chapter develops a finite-volume solver for the steady two-dimensional heat conduction equation with constant thermal conductivity. In the process, we introduce two new C++ concepts. First, we introduce C++ **vectors**, which are dynamic arrays whose size can be determined at run time. Second, we demonstrate how to save data to file for post-processing. The end-of-chapter homework guides you through the process of assembling the sparse matrix problem and verifying the solver using the method of manufactured solutions. We then use the solver to simulate the temperature field in a heat fin and explore the effects of inconsistent boundary conditions at corners of a numerical domain.

A key lesson of this chapter is that a simulation can appear physically reasonable at first glance, and yet contain a singularity that corrupts the data. In the case of the heat fin, we find there is a singularity that is not visible in a color plot of the temperature field, yet manifests itself as extremely slow convergence with grid refinement. This illustrates why it is essential to always investigate the sensitivity of simulation results to the grid resolution. Otherwise, issues like these go unnoticed, and inaccurate results are reported with false confidence.

3.1 Motivating application: Heat fin with inconsistent boundary conditions

CFD engineers are often asked to simulate a particular component or process within a larger complex system. When the surrounding system is too large to simulate directly, the engineer must truncate the computational domain to a manageable size and approximate the influence of the surrounding system through boundary conditions. One common pitfall in this process is the problem of inconsistent boundary conditions at corners, where two boundaries meet and imply different values at the same point.

To demonstrate this issue, we develop a C++ code that simulates steady heat conduction in a heat fin that dissipates heat from an engine block to the surrounding air, as sketched in Fig. 3.1. We assume 2D heat conduction in the $x - y$ plane, and we limit the computational domain to the fin itself, defined by $x \in [0, L_x]$ and $y \in [0, L_y]$. We also use a uniform finite-volume grid with constant cell dimensions Δx and Δy . The temperature field $T(x, y)$ is governed by the steady heat conduction equation



$$\int_{CS} k(\nabla T \cdot \mathbf{n}) dA + \int_{CV} q(\mathbf{x}) dV = 0, \quad (3.1)$$

where k is a constant thermal conductivity, and $q(\mathbf{x})$ is a volumetric heat source. Though the motivating problem has no heat source, we include $q(\mathbf{x})$ because it is used to verify our code using the method of manufactured solutions. We model the adjacent engine block by applying a constant block temperature T_b along the southern boundary ($y = 0$) of the domain,

$$T|_{y=0} = T_b. \quad (3.2)$$

Our engineering objective is to compute the rate of heat transfer (per unit depth in the z -direction) from the engine block into the fin,

$$\dot{Q} = \int_0^{L_x} \left(-k \frac{\partial T}{\partial y} \Big|_{y=0} \right) dx, \quad (3.3)$$

for the fixed fin dimensions $L_x = 3$ mm and $L_y = 15$ mm, thermal conductivity $k = 200$ W/m/K (aluminum), engine block temperature $T_b = 150^\circ\text{C}$ (423.15 K), and air temperature $T_{air} = 25^\circ\text{C}$ (298.15 K). To demonstrate how a modeling choice can produce inconsistent boundary conditions, suppose we model the surrounding air by applying a constant temperature T_{air} on the remaining three boundaries of the fin,¹

$$T|_{x=0} = T|_{x=L_x} = T|_{y=L_y} = T_{air}. \quad (3.4)$$

Assuming $T_b > T_{air}$, these boundary conditions are inconsistent at the two bottom corners, where the southern boundary meets the side faces.

¹Though a convective boundary condition would be more physically accurate, we deliberately use a fixed temperature condition to illustrate the impact of a corner inconsistency.

3.2 Code overview

File (3.1) below outlines the suggested structure of the code we build in this chapter.

File 3.1: Overall structure of the solver.

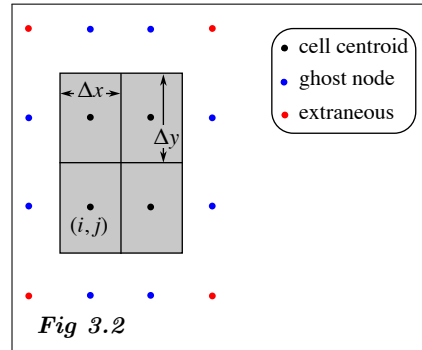
```
1 int main()
2 {
3     // --- fixed inputs (defined as const) ---
4     const double Lx    = 0.003;    // fin width (m)
5     const double Ly    = 0.015;    // fin height (m)
6     const double k      = 200.0;    // conductivity (W/m/K)
7     const double Tb     = 423.15;   // engine block temp (K)
8     const double Tair   = 298.15;   // air temp (K)
9
10    // --- variable inputs (read from terminal) ---
11    int Nx, Ny;           // grid resolution
12
13    // --- read, verify, and display inputs ---
14    ...
15
16    // --- save grid data to file ---
17    ...
18
19    // --- build 2D grid ---
20    ...
21
22    // --- build sparse matrix ---
23    ...
24
25    // --- build right-hand-side vector ---
26    ...
27
28    // --- solve linear system ---
29    ...
30
31    return 0;
32 }
```

Within `main()`, File (3.1) declares the fixed inputs (L_x , L_y , k , T_b , and T_{air}) as `const double` variables to protect them from accidental modification. The grid resolution (N_x and N_y) is read from the terminal. Before proceeding to the next sections, we suggest you first modify File (3.1) to perform the following steps:

1. Read N_x and N_y from the terminal. Display the input values back to the terminal so the user can confirm they were read correctly.
2. To ensure a minimum grid resolution of $N_x = N_y = 2$, verify that N_x and N_y are both greater than or equal to 2. If either condition is not satisfied, display an error message using `std::cerr` and exit the program using `return EXIT_FAILURE`, as discussed in Section 1.10.

3.3 C++ vectors

Once the input parameters have been read from the terminal, the next step is computing the grid data. For this problem, we require only the cell dimensions (Δx and Δy) and the coordinates of the grid points, sketched in Fig. 3.2 for the case $N_x = N_y = 2$. The grid points include the cell centroids (black dots) and ghost nodes (blue and red dots), giving a total of $(N_x + 2) \times (N_y + 2)$ grid points. The ghost nodes shown in blue are used to apply boundary conditions, while those shown in red require special treatment in later sections. In what follows, we assume the cell dimensions are computed as



```
// --- build grid ---
double dx = Lx/Nx;
double dy = Ly/Ny;
```

We store the coordinates of the grid points as two 1D arrays named **x** and **y**, where **x** stores the $N_x + 2$ x -coordinates, and **y** stores the $N_y + 2$ y -coordinates. The coordinates of the cell centroid labeled (i, j) in Fig. 3.2 are given by

$$x_i = \mathbf{x}[i], \quad y_j = \mathbf{y}[j]. \quad (3.5)$$

Throughout this textbook, we consistently index x -coordinates using $i \in [0, N_x + 1]$ and y -coordinates using $j \in [0, N_y + 1]$.

Because N_x and N_y are not known until run time, we cannot define **x** and **y** as C-style arrays.² Instead, we use a C++ tool called a *vector*, a dynamic array that behaves much like a regular array but automatically manages its own memory and keeps track of its size.

3.3.1 A basic demonstration

To demonstrate the basic use of C++ vectors, File (3.2) below computes the **x** array. For brevity, we omit the input and grid spacing statements from File (3.1).

File 3.2: Using a C++ vector to build the **x** array.

```
1 #include <vector> // needed for vectors
2
3 int main()
4 {
5
6     // allocate x vector
7     std::vector<double> x(Nx+2);
8
9     // compute x coordinates
10    for (int i = 0; i <= Nx+1; ++i)
11    {
12        x[i] = -0.5*dx + i*dx;
13    }
```

²Recall that the size of C-style arrays must be known at compile time.

File (3.2) performs the following operations:

- Line 1 includes the `vector` header, which is required to use C++ vectors:

```
1 #include <vector> // needed for vectors
```

- Line 7 declares a vector named `x` using the syntax:

```
std::vector<type> name(size);
```

Here, `type` is the data type of the vector entries, and `size` is the number of elements. For example, a vector of a vector of 10 integers named `indices` is declared as

```
std::vector<int> indices(10);
```

- Lines 10–13 use a `for`-loop to compute the x -coordinates and store them in `x`. The first and last elements (`x[0]` and `x[Nx+1]`) store the x -coordinates of the ghost nodes on the west and east boundaries, respectively. The remaining elements store the x -coordinates of the cell centroids.

Note: We leave it to the reader to build the `y` array in the same manner.

3.3.2 Vector member functions

Unlike C-style arrays, which are fixed in size and have no built-in operations, C++ vectors can dynamically change their size, and come with a suite of useful *member functions*. These functions perform common tasks and are accessed using the dot (`.`) notation. For example, to determine the number of elements stored in a vector, we can use the `size()` function as shown below:

File 3.3: Using the `size()` member function of a vector.

```
1 std::vector<double> x(10); // a vector with 10 elements
2 std::cout << "Size of x = " << x.size() << std::endl;
```

In the excerpt above, `x.size()` returns the number of elements stored in the vector `x`.

Vectors provide many member functions, but here we list only a few in Table 3.1. The functions at the top access or read elements without changing the number of elements stored in the vector. The functions below them modify the vector in some way, either by adding, removing, or resizing elements. For our limited scope, the excerpt in File (3.4) below demonstrates that you can declare an empty vector with no size, and then grow the vector dynamically by appending elements using the `push_back` function:

File 3.4: Appending elements to a vector using `push_back`.

```
1 // declare empty vector
2 std::vector<int> numbers;
3
4 // dynamically add elements
5 for (int i = 0; i < 5; ++i)
6 {
7     numbers.push_back(i*10);
8 }
```

Function	Description
<code>size()</code>	Returns the number of elements in the vector.
<code>front()</code>	Returns first element.
<code>back()</code>	Returns last element.
<code>at(index)</code>	Returns element at <code>index</code> , with bounds checking.
<code>push_back(value)</code>	Adds <code>value</code> to the end of the vector.
<code>insert(pos, value)</code>	Inserts <code>value</code> before the position <code>pos</code> .
<code>erase(pos)</code>	Removes the element at position <code>pos</code> .
<code>clear()</code>	Removes all elements from the vector.
<code>resize(n)</code>	Changes the number of elements stored.

Table 3.1: Some useful C++ vector member functions.

Following the `for`-loop, the vector `numbers` has a size of 5 and contains the elements

0, 10, 20, 30, 40.

3.4 Writing data to a file

After building the grid, we save the grid coordinates to a file for later post-processing. File 3.5 demonstrates the process below, where for brevity we only show the relevant lines of code.

File 3.5: Writing grid coordinates to file.

```

1 #include <fstream> // required to write data to file
2
3 int main()
4 {
5
6     // open file for writing
7     std::ofstream xfile;
8     xfile.open("x.dat");
9
10    // write coordinates as a single column
11    for (int i = 0; i < Nx+2; i++)
12    {
13        xfile << x[i] << "\n";
14    }
15    xfile.close();
16
17 }
```

File 3.5 performs the following operations:

- Lines 7–8 prepare the file `x.dat` for writing:

```

7  std::ofstream xfile;
8  xfile.open("x.dat");

```

Line 7 creates a variable named `xfile` of type `ofstream`, a C++ datatype used for writing to files. Line 8 opens the file `x.dat` for writing. Note that both of these operations require the `<fstream>` header included on line 1.

- Lines 11–14 use a `for`-loop and the insertion operator `<<` to write the elements of the `x` vector to the file. The syntax is similar to what we have used previously to print data to the terminal:

```
std::cout << value;
```

Now, however, we replace `std::cout` with

```
xfile << value;
```

Each iteration of the `for`-loop writes one coordinate value to `x.dat`, followed by a new-line.

- Line 15 closes the file:

```
xfile.close();
```

Closing the file frees system resources and ensures all data has been written correctly.

Note: We leave it to the reader to similarly save the y -coordinates of the grid nodes to a file. To verify your code, we recommend running your code for a small resolution, and verifying the data in the files `x.dat` and `y.dat` is correct.

3.5 Homework

Problem 1: Building the sparse matrix

We now have the necessary C++ tools to compute and store the nonzero entries of the sparse matrix $\mathbf{A}$ defined in section 3.5.2 of the theory textbook. For that, you are asked to complete the starter code in File (3.6) below. Because the number of nonzero entries in a sparse matrix is often not known in advance, the code stores them dynamically in three C++ vectors named `ROW`, `COL`, and `VAL`, using `push_back` to append each new entry as it is assembled.

To help you get started, the starter code already computes the nonzero entries corresponding to the Dirichlet boundary conditions. Your task is to complete the double `for`-loop that applies the 5-point stencil on the interior nodes, as well as to set the extraneous nodes to zero. Note that lines 65–69 of File (3.6) set the coefficients of the 5-point stencil as defined in section 3.2.4 of the theory textbook.

Note about indices: Pay attention to the limits of the `for`-loops and the formula `row = j*Nx2 + i`. These differ from the Matlab codes demonstrated in section 3.5.2 of the theory textbook. The difference arises due to the zero-based indexing used in C++.

File 3.6: An incomplete C++ code that builds the sparse **A** matrix.

```
1 int main()
2 {
3
4     // --- build matrix ---
5     int row;
6     int Nx2  = Nx + 2;
7     int Ny2  = Ny + 2;
8
9     std::vector<int>    ROW, COL;
10    std::vector<double> VAL;
11
12    // south/north boundary conditions
13    for (int i = 1; i <= Nx; i++)
14    {
15        // south
16        row = i;
17
18        ROW.push_back(row);
19        COL.push_back(row);
20        VAL.push_back(0.5);
21
22        ROW.push_back(row);
23        COL.push_back(row + Nx2);
24        VAL.push_back(0.5);
25
26        // north
27        row = (Ny+1)*Nx2 + i;
28
29        ROW.push_back(row);
30        COL.push_back(row);
31        VAL.push_back(0.5);
32
33        ROW.push_back(row);
34        COL.push_back(row-Nx2);
35        VAL.push_back(0.5);
36    }
37
38    // east/west boundary conditions
39    for (int j = 1; j <= Ny; j++)
40    {
41        // west
42        row = j*Nx2;
43
44        ROW.push_back(row);
45        COL.push_back(row);
46        VAL.push_back(0.5);
```



```

47
48     ROW.push_back(row);
49     COL.push_back(row + 1);
50     VAL.push_back(0.5);
51
52     // east
53     row = (j+1)*Nx2 - 1;
54
55     ROW.push_back(row);
56     COL.push_back(row);
57     VAL.push_back(0.5);
58
59     ROW.push_back(row);
60     COL.push_back(row-1);
61     VAL.push_back(0.5);
62 }
63
64 // 5-point stencil
65 double ae = k*dy/dx;
66 double aw = k*dy/dx;
67 double an = k*dx/dy;
68 double as = k*dx/dy;
69 double ap = ae + aw + an + as;
70
71 for (int i = 1; i <= Nx; i++)
72 {
73     for (int j = 1; j <= Ny; j++)
74     {
75         ...
76         ...
77     }
78 }
79
80 // apply conditions on extraneous nodes
81 ...
82 ...
83 }

```

Problem 2: Building the right-hand side vector

Next, complete the starter code in File (3.7) below, which constructs the right-hand side vector **b**. The code applies the required boundary conditions and source term to force the manufactured solution

$$T_{ex} = \sin(x) \sin(y). \quad (3.6)$$

The code stores the right-hand-side in a vector named **B**. The vector is allocated on line 8 using the syntax

```
std::vector<double> B(Ntot, 0.0);
```

This syntax initializes all elements of **B** to 0. The starter code already applies the boundary conditions. Your task is to finish populating the vector for all interior nodes. Note that the starter code requires the `<cmath>` header.

File 3.7: Starter code for building the right-hand side vector **b**. Complete the interior nodes.

```
1 int main()
2 {
3
4     // initialize vector to zeros
5     int Ntot = Nx2*Ny2; // total nodes
6     std::vector<double> B(Ntot, 0.0);
7
8     // south/north boundary conditions
9     for (int i = 1; i <= Nx; i++)
10    {
11        // south
12        row    = i;
13        B[row] = std::sin(x[i])*std::sin(0);
14
15        // north
16        row    = (Ny+1)*Nx2 + i;
17        B[row] = std::sin(x[i])*std::sin(Ly);
18    }
19
20    // east/west boundary conditions
21    for (int j = 1; j <= Ny; j++)
22    {
23        // west
24        row    = j*Nx2;
25        B[row] = std::sin(0)*std::sin(y[j]);
26
27        // east
28        row    = (j+1)*Nx2 - 1;
29        B[row] = std::sin(Lx)*std::sin(y[j]);
30    }
31
32    // internal nodes
33    for (int i = 1; i <= Nx; i++)
```

```

34 {
35     for (int j = 1; j <= Ny; j++)
36     {
37         ...
38     }
39 }
40 }

```

Problem 3: Solving the linear system

To solve the linear system $\mathbf{AT} = \mathbf{b}$, paste the code in File (3.8) below into your source file. The code takes the vectors ROW, COL, VAL, and B and uses the *Eigen* package to solve the system using a sparse LU decomposition. It then saves the solution vector $\mathbf{T}$ in a file called T.dat. The vector is saved as a column, using the same ordering as the global matrix $\mathbf{A}$.

Note: The code in File (3.8) requires several C++ concepts that we have not yet covered. For now, you can simply paste the code into your source file and treat it as a black box that takes the vectors ROW, COL, VAL, and B as input and writes the solution to T.dat.

Required headers: To use the code below, add the following three headers at the top of your source file:

```

#include <cstdlib>           // required for EXIT_FAILURE
#include <Eigen/Sparse>      // required to define sparse A matrix
#include <Eigen/Dense>       // required to define right-hand-side
#include <Eigen/SparseLU>    // required for sparse LU

```

Eigen setup: Download Eigen 5.0.0 from <https://libeigen.gitlab.io> and extract it to a folder on your computer. To make it available to your compiler, specify the path to the Eigen folder when compiling. For example, using g++:

```
g++ -I/path/to/eigen mycode.cpp -o mycode
```

The -I/path/to/eigen flag tells the compiler where to find the Eigen headers.

File 3.8: Code to solve the linear system $\mathbf{AT} = \mathbf{b}$ using Eigen and save the solution to T.dat

```

1 int main()
2 {
3     // --- solve linear system ---
4
5     // load matrix
6     Eigen::SparseMatrix<double> A(Ntot, Ntot);
7     std::vector<Eigen::Triplet<double>> triplets;
8     triplets.reserve(VAL.size());
9     for (size_t k = 0; k < VAL.size(); ++k)
10    {
11        triplets.emplace_back(ROW[k], COL[k], VAL[k]);
12    }

```

```

13  A.setFromTriplets(triplets.begin(), triplets.end());
14
15  //load right-hand-side
16  Eigen::VectorXd RHS(Ntot);
17  for (int i = 0; i < Ntot; ++i)
18  {
19      RHS[i] = B[i];
20  }
21
22  // compute the LU factorization of A
23  Eigen::SparseLU<Eigen::SparseMatrix<double>> solver;
24  solver.compute(A);
25  if(solver.info() != Eigen::Success)
26  {
27      std::cerr << "Error: LU decomposition failed";
28      return EXIT_FAILURE;
29  }
30
31  // solve
32  Eigen::VectorXd T = solver.solve(RHS);
33  if(solver.info() != Eigen::Success)
34  {
35      std::cerr << "Error: Eigen could not solve";
36      return EXIT_FAILURE;
37  }
38
39  // write solution to file
40  std::ofstream Tfile;
41  Tfile.open("T.dat");
42  for (int i = 0; i < Ntot; i++)
43  {
44      Tfile << T[i] << "\n";
45  }
46  Tfile.close();
47
48 }

```

N_x	8	12	16	24	32	48	64
N_y	12	18	24	36	48	72	96
E_∞	0.00873	0.00386	-	-	-	-	-
$\dot{Q}_{err}$	0.00944	0.00419	-	-	-	-	-

Table 3.2: Grid resolutions and sample results for the convergence study. Remember to set $L_x = 2.5$ and $L_y = 3.5$.

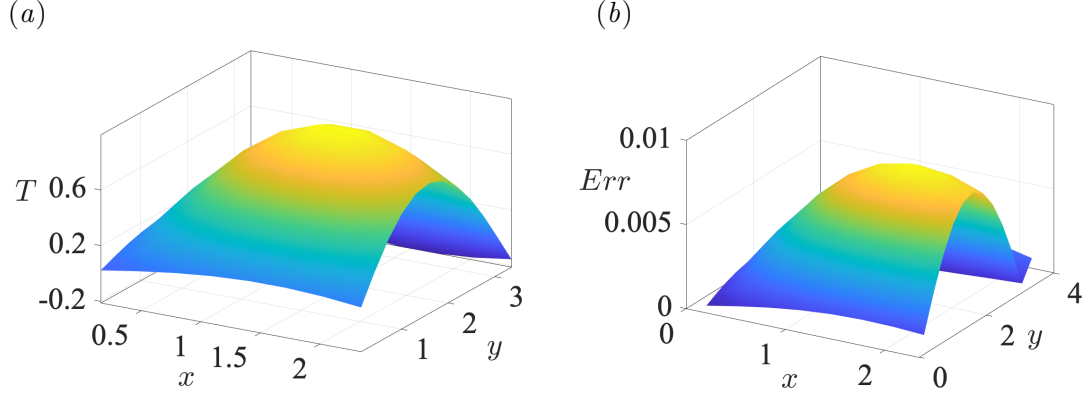


Fig 3.3: Results for manufactured solution when $(N_x, N_y) = (8, 12)$: (a) Numerical result for T , (b) $Err(x, y)$

Problem 4: Post-Processing and Verification

Note: For this problem, temporarily modify your code to the set domain dimensions to $L_x = 2.5$ and $L_y = 3.5$. These dimensions are more appropriate for the manufactured solution used to verify the code.

Assuming your code compiles and runs, write a post-processing routine using Matlab/Python that does the following:

- Loads the files `x.dat`, `y.dat`, and `T.dat`.
- Generates three surface plots showing the following quantities on the internal nodes (excluding ghost nodes):
 1. the numerical solution
 2. the exact manufactured solution $T_{ex} = \sin(x) \sin(y)$.
 3. the local error $Err(x, y)$.
- Computes the global infinity-norm error E_∞ .

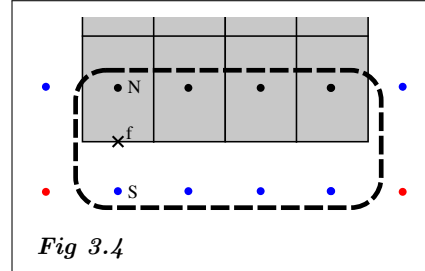
To verify your C++ implementation, run your C++ code for each of the grid resolutions listed in Table 3.2. For each resolution, inspect the plots generated by your post-processing

routine, and record the global error E_∞ . For comparison, our expected results for E_∞ are included for $N_x = 8$, and 12. Figure 3.3 also shows expected for the simulated temperature field T (panel *a*) and the error $Err(x, y)$ (panel *b*).

If your results behaves as expected, generate a log-log plot of E_∞ vs. N_x to confirm that your code produces the expected second-order spatial accuracy. Note that the resolutions in Table 3.2 use different values for N_x and N_y , but maintain a fixed ratio $N_y/N_x = 1.5$. This tests your code for a common programming error in which the code accidentally uses N_x where N_y is required, or vice versa.

Problem 5: Computing the heat transfer rate $\dot{Q}$

Once you have verified the spatial accuracy of your code, modify your post-processing routine so that it also compute of the heat transfer rate $\dot{Q}$. For that, Fig. 3.4, shows a row of cells along the southern boundary when $N_x = 4$. The area centroid of the left-most cell face is marked with a $\times$ and labeled f . Explain why we can approximate the contribution of $\dot{Q}$ through this single face as



$$\Delta \dot{Q} \approx -k \left(\frac{T_N - T_S}{\Delta y} \right) \Delta x, \quad (3.7)$$

where T_N and T_S are the temperatures at the adjacent cell centroid and ghost node.

Using the approximation above, compute $\dot{Q}$ and verify it by computing the relative error

$$\dot{Q}_{err} = \frac{\|\dot{Q} - \dot{Q}_{ex}\|_\infty}{\|\dot{Q}_{ex}\|_\infty}, \quad (3.8)$$

where $\dot{Q}_{ex}$ is the exact answer produced by the manufactured solution,

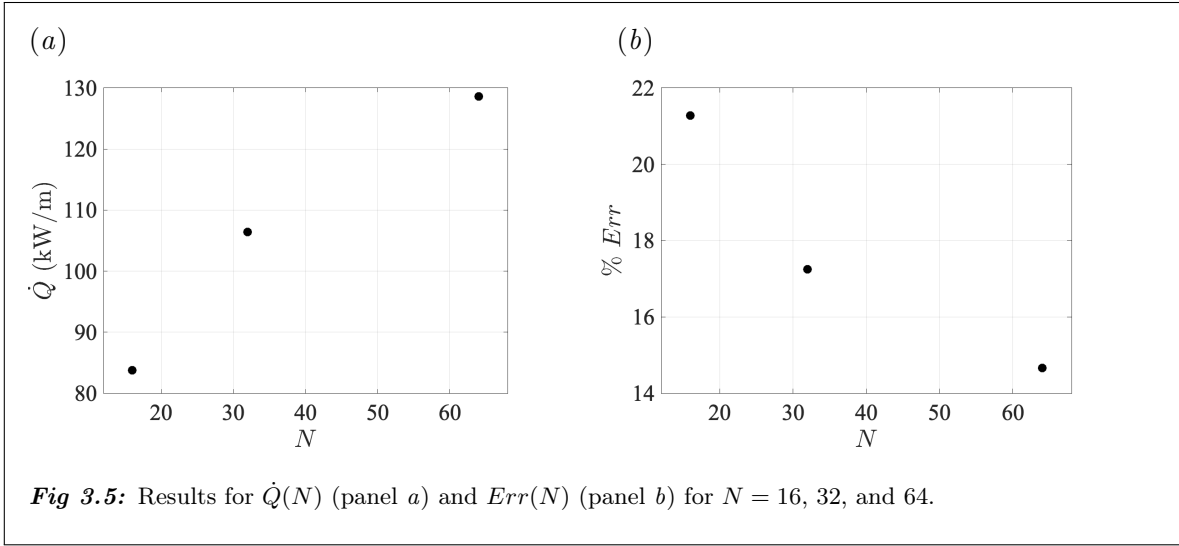
$$\dot{Q}_{ex} = -k[1 - \cos(L_x)]. \quad (3.9)$$

Table 3.2 shows expected results for $N_x = 8$, and 12.

Problem 6: Simulating the heat fin

Having verified the solver using the method of manufactured solutions, we now apply it to simulate the actual heat fin described in Section 4.1. Copy your C++ code to a new file, so you don't lose the code that verifies your code, and make the following modifications:

1. Remove the artificial volumetric heat source $q(\mathbf{x})$ from the right-hand-side vector $\mathbf{b}$, apply the physical temperature boundary conditions, and set $L_x = 0.003$ m and $L_y = 0.015$ m.
2. Write a post-processing routine (in Matlab or Python) that:
 - (a) Generates a 3D surface plot of T (in $^\circ\text{C}$) vs. x and y , excluding ghost nodes.
 - (b) Generates a 2D color plot of T (in $^\circ\text{C}$) vs. x and y , with a colorbar.
 - (c) Computes $\dot{Q}$.
3. Run your code and post-processing routine for a grid resolution of $N_x = N_y = 64$. Display and interpret your results. Does the simulated temperature field look physically reasonable?



Problem 7: Grid sensitivity of $\dot{Q}$

Before using our solver to draw engineering conclusions, we must assess how sensitive the predicted heat transfer rate $\dot{Q}$ is to the grid resolution. To do so, we adopt the following procedure. Setting $N_x = N_y = N$, we estimate the percentage relative error in $\dot{Q}$ due to grid resolution by comparing results on two successive grids:

$$Err(N) = \frac{|\dot{Q}(N) - \dot{Q}(2N)|}{|\dot{Q}(2N)|} \times 100\%. \quad (3.10)$$

In the above, $Err(N)$ is the estimated error for a grid resolution N , and $\dot{Q}(N)$ is computed using resolution N . For example, the error at $N = 32$ is estimated as

$$Err(32) = \frac{|\dot{Q}(32) - \dot{Q}(64)|}{|\dot{Q}(64)|} \times 100\%. \quad (3.11)$$

Using the error estimate above, perform the following:

1. Perform simulations for $N = 32, 64, 128, 256, 512, 1024$, and make a plot of $\dot{Q}(N)$ vs. N . Figure 3.5(a) demonstrates expected results for $N \leq 64$.

Note: The simulation at $N = 1024$ may take several minutes, as the current implementation of the Eigen solver has not been optimized for large systems. If it takes too long on your computer, you can stop at $N = 512$. However, that means you can only estimate the error up to $N = 256$.

2. Compute $Err(N)$ for $32 \leq N \leq 512$, and generate a plot of Err vs. N . Figure 3.5(b) demonstrates expected results for $N \leq 64$. You should find that $Err(N)$ never falls below 1 %.
3. To estimate the grid resolution N^* that achieves a relative error of 1%, fit your computed data for $Err(N)$ to a power law of the form

$$Err = aN^b \quad (3.12)$$

and solve for

$$N^* = \left(\frac{1}{a}\right)^{1/b}. \quad (3.13)$$

Comment on whether this is a practical grid resolution.

Problem 8: (optional)

Though your color plots of $T(x, y)$ may appear to be physically reasonable, you should find that $\dot{Q}$ converges very slowly to a reasonable error with increasing grid resolution. To understand why, this optional problem asks you to compute and plot the gradient $\partial T / \partial y$ along the southern boundary of the domain,

$$\left. \frac{\partial T}{\partial y} \right|_{y=0}. \quad (3.14)$$

Figure 3.6 shows the results for $x \leq 0.3$ mm (near the southwest corner of the domain) when $N = 64, 128, 256, 512$, and 1024. The results show that with increasing N , a singularity forms at $x = 0$, where $\partial T / \partial y \rightarrow -\infty$ as $N \rightarrow \infty$. Why might this singularity explain the poor convergence of $\dot{Q}$ with N ?

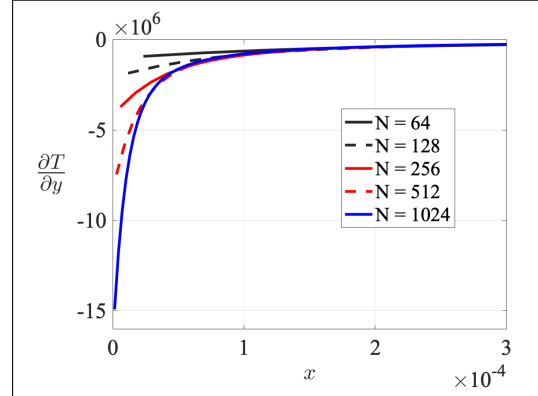


Fig 3.6: The gradient $\partial T / \partial y$ along the boundary $y = 0$ for $N = 64, 128, 256, 512$, and 1024. We only show results for $0 \leq x \leq 0.3$ mm.